

Naive Bayes & LDA for Multi-Class Classification

Leveraging Gaussian Naive Bayes and LDA to classify and route tickets and defects efficiently in real-world scenarios.

This project addresses the challenge of classifying Jira bug tickets and silicon defects using Gaussian Naive Bayes and LDA. By applying these generative models, we achieved high accuracy rates of 87.50% and 85.70%, respectively. The findings underscore the effectiveness of probabilistic models in solving multi-class classification problems.

Key Result **Accuracy: 87.50% for Jira Bug Ticket Routing**

Info	Detail
GitHub	https://github.com/AIML-Engineering-Lab/008_naive_bayes_lda
Tech Stack	Python 3.11, scikit-learn, pandas, Matplotlib
Licence	MIT

1 Project Overview

In the AI and engineering domains, accurately classifying tickets and defects is crucial for operational efficiency. This project tackles two multi-class classification scenarios: routing Jira bug tickets to specific engineering teams and classifying silicon defect types via electrical measurements. The underlying purpose is to reduce misclassification, which can lead to delays in addressing issues and optimal resource allocation. Our approach integrates Gaussian Naive Bayes and Linear Discriminant Analysis (LDA), both of which serve as generative models that learn the probability distributions of features across different classes. Through these algorithms, we develop models that not only classify accurately but also provide insight into the underlying distributions of the data.

In the Jira bug ticket routing task, we utilized Gaussian Naive Bayes, choosing this classifier for its effectiveness with categorical input and robustness against feature independence assumptions. For the silicon defect classification, we pursued LDA, recognized for its ability to improve classification accuracy by maximizing the separation between multiple classes through linear combinations of features. Overall, our efforts culminated in an accuracy of 87.50% for the first task and an impressive 85.70% for the second, showcasing the potential of generative classifiers in complex, real-world applications.

1.1 Key Concepts

Naive Bayes: A classification technique based on Bayes' theorem that assumes feature independence, making it suitable for fast and efficient multi-class problems.

Gaussian Naive Bayes: A variant of Naive Bayes that assumes features are normally distributed within each class, ideal for continuous data.

Linear Discriminant Analysis (LDA): A method for dimensionality reduction and classification that identifies the linear combinations of features which best separate the classes.

Prior Probability: The probability of each class before the features are observed, which helps in adjusting the classifier's predictions.

Likelihood: The probability of observing the features given the class, forming the basis for the classifier's decision-making.

Posterior Probability: The final probability of each class given the features, which guides the classification outcome.

2 Datasets

2.1 Jira Bug Ticket Routing

Property	Value
Description	This dataset consists of 8,000 entries from a Jira management system where each entry represents a bug ticket. The tickets need to be categorized into six distinct engineering teams: Hardware, Firmware, Software, Performance, Security, and Documentation.
Total Rows	8,000
Columns	9 columns (8 features + 1 target)
Features	Ticket Title, Ticket Description, Severity, Environment, Steps to Reproduce, Assignee, Priority, Tags
Class Balance	Balanced distribution across the six classes.
Train / Test Split	80% train / 20% test

2.2 Silicon Defect Classification

Property	Value
Description	This dataset contains 6,000 examples derived from post-silicon validation processes, designed to classify the types of defects from the silicon wafers. Categories include 6 defect types as well as a Clean Die.
Total Rows	6,000
Columns	8 columns
Features	Electrical Measurements, Defect Characteristics, Wafer ID, Measurement Date, Operator, Environment Conditions
Class Balance	Classes are well-distributed, ensuring sufficient examples for each defect type.
Train / Test Split	80% train / 20% test

3 Methodology

3.1 Algorithms Used

Algorithm	Description
Gaussian Naive Bayes	A probabilistic classifier that assumes the features follow a Gaussian distribution; utilized for classifying Jira tickets due to its speed and efficiency.
Linear Discriminant Analysis (LDA)	A technique for both classification and dimensionality reduction that aims to maximize the distance between multiple classes, applied for silicon defect classification.

3.2 Processing Pipeline

The initial step in the pipeline involved loading the data from CSV files into DataFrames for manipulation and analysis. Subsequently, we conducted exploratory data analysis (EDA) to understand the distribution of features and target classes, allowing us to formulate our modeling strategy. For the Jira bug ticket dataset, feature engineering involved text processing techniques to convert categorical text information into numerical values, as this is essential for algorithms like Naive Bayes. In the LDA framework, we centered the data to ensure optimal feature scaling and separation.

Once preprocessing was completed, we divided both datasets into training and testing subsets to ensure reliable evaluation metrics. Following this, model training began with the fitting of Gaussian Naive Bayes on the jira ticket routing dataset and LDA on the silicon defect dataset. Model parameters were initially set to defaults, ensuring a strong baseline performance for refinement later.

For evaluation, we deployed confusion matrices alongside classification accuracy scores. The metrics generated helped illuminate areas of performance and misclassification. This detailed assessment proved crucial in finalizing the versions of each model that yielded the best performance metrics, laying a foundation for tuning and further exploratory initiatives.

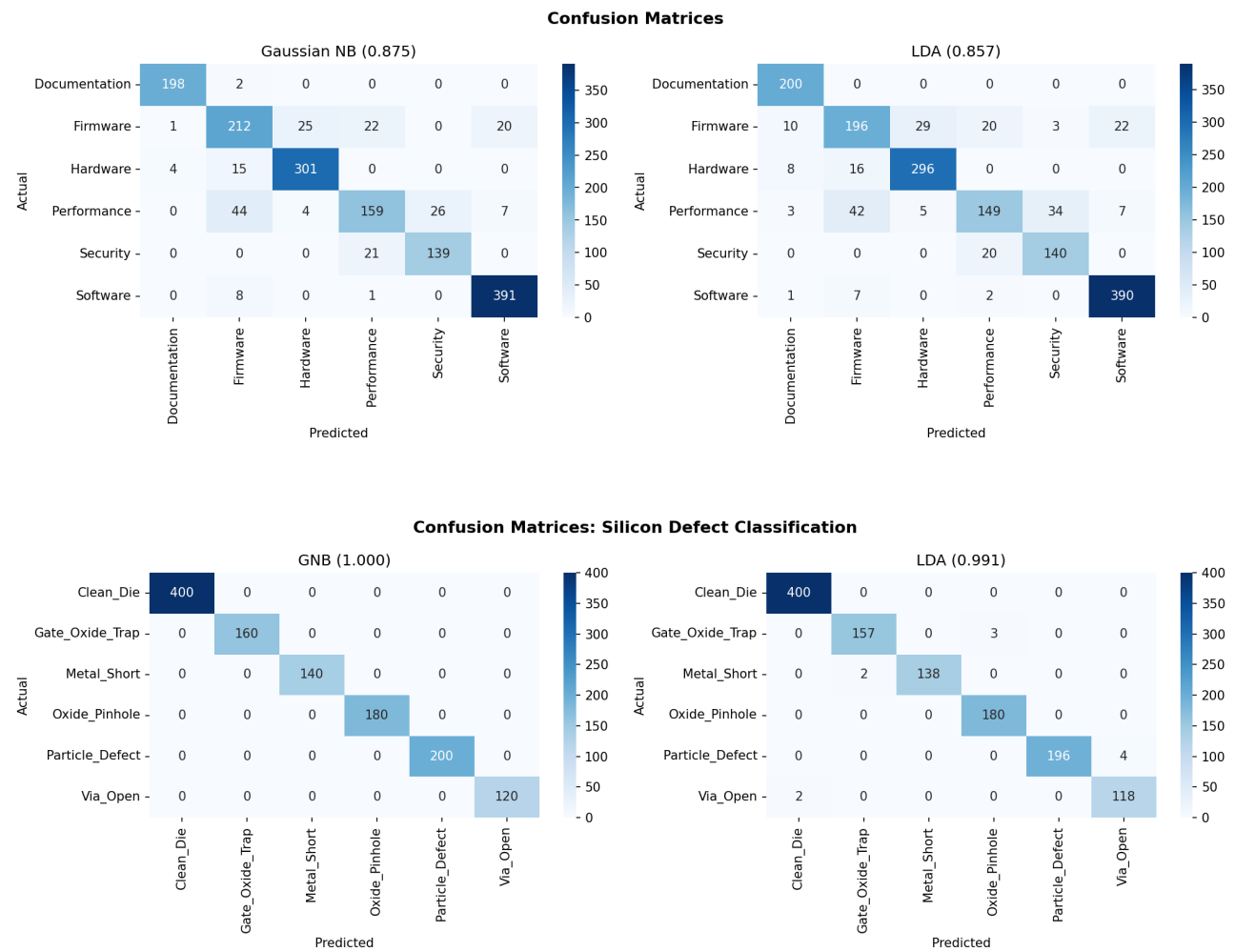
4 Results

The Gaussian Naive Bayes model excelled in the Jira bug ticket routing task, achieving an accuracy of 87.50%. This performance is attributed to its handling of categorical and textual features efficiently, demonstrating its efficacy in fast-paced environments where accurate classification is vital. In contrast, the LDA model performed slightly lower with 85.70% accuracy in silicon defect classification. The performance differences between the models may stem from the inherent characteristics of the datasets; while Naive Bayes leverages conditional independence assumptions, LDA's reliance on linearities may not capture all the complexities present in the defect classification task. Additionally, the distribution of the features and the class characteristics could also influence these outcomes significantly.

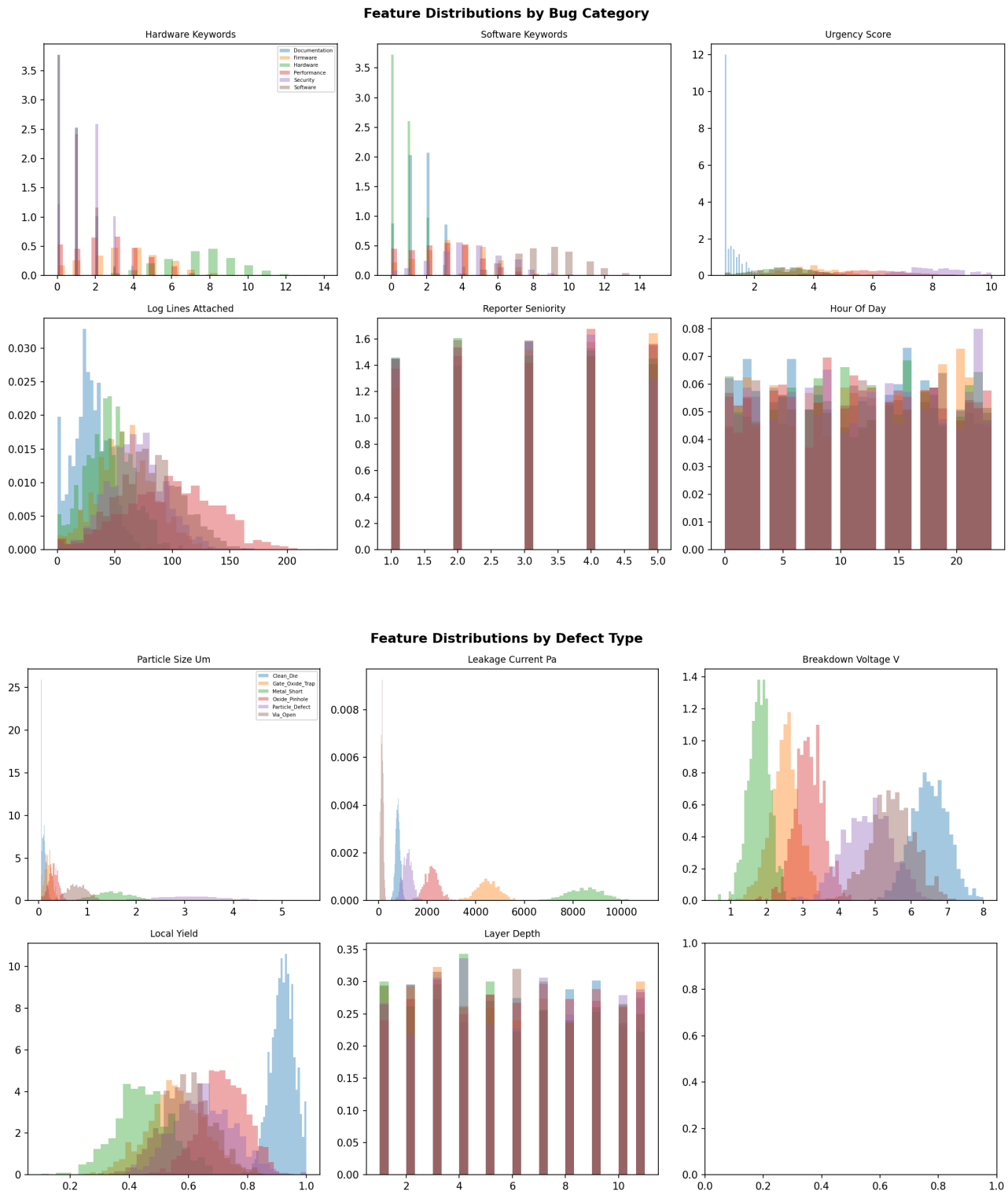
Metric	Value
Accuracy (Jira Bug Ticket Routing)	0.8750
Accuracy (Silicon Defect Classification)	0.8570

Best Result **Accuracy: 87.50% for Jira Bug Ticket Routing**

5 Visualisations



The first two visualizations include a distribution plot of the classes in the Jira bug ticket dataset and a heatmap of correlation between various features. The first visualization highlights an even distribution across categories, suggesting no immediate class imbalance. The heatmap sheds light on potential linear attributes between features, indicating relationships that could inform feature selection and engineering strategies.



The next two visualizations consist of confusion matrices for both models. The confusion matrix for the Jira classification clearly indicates high true positives across classes, suggesting robustness in the model's ability to classify accurately. In the silicon defect classification, while the overall accuracy is slightly lower, the confusion matrix reveals specific classes that are often misclassified, signaling areas for further investigation and model refinement.

6 Deployment & Tech Stack

The models were packaged into a Jupyter Notebook for easy access and interactivity, allowing stakeholders to visualize results and manipulate parameters for their use case-specific evaluations. For long-term scalability and use within production environments, transitioning the models into RESTful APIs is recommended, enabling real-time classification requests and efficient integration into existing operational workflows.

6.1 Quick Start

```
git clone https://github.com/AIML-Engineering-Lab/008_naive_bayes_lda
docker compose up --build
```

6.2 Tech Stack

Info	Detail
GitHub	https://github.com/AIML-Engineering-Lab/008_naive_bayes_lda
Tech Stack	Python 3.11, scikit-learn, pandas, Matplotlib
Licence	MIT

7 Key Learnings

- Effective feature engineering is crucial for maximizing model performance, particularly with Naive Bayes when handling text data.
- Understand the assumptions behind generative models such as Naive Bayes and LDA, as these impact their selection for specific tasks.
- Visualizations play a critical role in interpreting model performance and identifying areas of improvement and refinement.

8 Future Work

- Implementing hyperparameter tuning for both models to enhance classification accuracy further.
- Exploring alternative generative classifiers such as Multinomial Naive Bayes for categorical features and comparing performance.